ACRME Complete Requirements Reference

This document consolidates **every requirement for the Azure Capacity Reservation Management Engine (ACRME)**,
spanning Functional Requirements (FR), Non-Functional Requirements (NFR), and Placement-Specific Requirements (R),
organized by category with evidence of coverage and design maturity.

Sources:
- `azure_cr_management_engine_design.md` (FR-1..8, NFR-1..7)
- `acrme_production_readiness_review_and_architecture.md` (Section 22 Must/Should/Could; cross-references to PRR)
- `acrme_requirements_traceability_review.md` (coverage matrix + blockers)
- `multi_region_placement_design.md` (Placement Requirements R1..R8, NFR-R1..R3)

> **Classification:** Principal Cloud Architect — System Design
> **Maturity:** Production Readiness Review Complete; POC-gated execution ready

---

Part 1 — Functional Requirements (FR-1 through FR-8)

FR-1 — Capacity Reservation Lifecycle Management

Manages the full CRUD lifecycle of Capacity Reservation Groups (CRGs) and Capacity Reservations (CRs) across
the Azure estate at `api-version=2024-03-01` or later.

ID	Requirement	Coverage	Implementation
FR-1.1	CRG create, read, update, delete via Azure Compute REST API at API version 2024-03-01+	✅ COVERED	`EPIC-02` (Provisioning Engine); API endpoints: `POST/GET/PATCH/DELETE /api/v1/crgs`
FR-1.2	CR create, read, update, delete within managed CRGs; support quantity modification without VM disruption	✅ COVERED	`EPIC-02` stories E02-S03, E02-S04; `PATCH /crgs/{id}/reservations/{cr_id}` accepts new quantity
FR-1.3	Zero-size reservation pattern: create CRs at `quantity=0`, increment as VMs associate	✅ COVERED	E02-S07; CapacityReservation.desired_quantity permits 0; P1 backlog item
FR-1.4	Track `allocated` (VMs currently associated) vs `quantity` (reserved capacity) independently; alert on overallocation	✅ COVERED	CapacityReservation entity; E05-S06 (Overallocation Detection); `GET .../utilization` API
FR-1.5	Enforce CR deletion sequencing: disassociate all VMs → delete all CRs → delete CRG; reject out-of-order deletes	✅ COVERED	E02-S05, E02-S06; cascade validation guards in Provisioning Service
FR-1.6	Support CR quantity reduction; enforce floor constraint (`quantity ≥ allocated`)	✅ COVERED	E02-S04; floor validation guard prevents reduction below allocated count

FR-1 Verdict: ✅ 6/6 covered — Fully Implemented

FR-2 — Capacity Reservation Sharing Management

Orchestrates the three-step RBAC setup for Provider-Consumer CRG sharing, managing subscription membership
and enforcing constraints.

ID	Requirement	Coverage	Implementation
FR-2.1	Manage `sharingProfile.subscriptionIds` : add, remove, list Consumer subscriptions on CRGs	✅ COVERED	`EPIC-04` (Sharing Management); `POST/GET/DELETE /crgs/{id}/consumers` ; updates ARM sharingProfile
FR-2.2	Orchestrate complete three-step RBAC setup for each Provider-Consumer pair: (1) grant Provider `deploy/action` on Consumer sub, (2) add Consumer to CRG sharingProfile, (3) grant Consumer identity read + `deploy/action` on CRG	✅ COVERED	E04-S02, E04-S06 (RBAC Orchestration); three-step sequence in SharingService; atomic guard
FR-2.3	Enforce 100-Consumer-per-CRG limit; return structured error with remediation when limit reached	✅ COVERED	E04-S01 (Constraint Validation); CRG.max_consumer_count = 100; HC-2 hard constraint
FR-2.4	Verify `Microsoft.Compute` provider registration in all Consumer subscriptions before completing sharing setup	✅ COVERED	E04-S07 (Provider Registration Validation); pre-gate in onboarding workflow
FR-2.5	Support safe unsharing: detect active Consumer VM associations before removal; require explicit force-override	✅ COVERED	E04-S03 (Safe Unsharing); check allocated count before removal; warn on forced unsharing
FR-2.6	Warn operators when forced unsharing is requested with active Consumer VMs citing silent hazard	✅ COVERED	E04-S03; alert `ForceUnsharingWithActiveVMs` (Warning severity)

ID	Requirement	Coverage	Implementation
FR-2.7	Maintain current and historical sharing profile state per CRG; record timestamps of add/remove operations	✅ COVERED	SharingRelationship entity; audit log; E04-S09 (History Tracking)

FR-2 Verdict: ✅ 7/7 covered — Fully Implemented

FR-3 — Availability Zone Mapping Management

Discovers, stores, and resolves logical-to-physical zone mappings across Provider-Consumer subscription pairs.

ID	Requirement	Coverage	Implementation
FR-3.1	Discover and store logical-to-physical zone mapping for every managed subscription in every managed region	✅ COVERED	`EPIC-09` (Zone Management); ZoneMappingService; E09-S01 (Discovery)
FR-3.2	Use Check Zone Peers API to compute cross-subscription zone equivalence for all Provider-Consumer pairs	✅ COVERED	E09-S02 (Cross-Subscription Equivalence); stores in zone mapping registry
FR-3.3	Maintain zone mapping registry: `(provider_sub_id, region, provider_zone) → [(consumer_sub_id, consumer_zone)]`	✅ COVERED	ZoneMappingRecord entity (Cosmos DB); E09-S01
FR-3.4	Automatically resolve correct Consumer logical zone when accepting VM deployment against shared CRG	✅ COVERED	E09-S03 (Zone Resolution); used in placement decision tree
FR-3.5	Detect and reject VM deployment where Consumer zone does not map to same physical zone as target CR	✅ COVERED	E09-S04 (Zone Mismatch Rejection); hard constraint HC-8 (ZONE_ALIGNMENT)
FR-3.6	Refresh zone mappings on configurable schedule and on new subscription onboarding event	✅ COVERED	E09-S05 (Scheduled Refresh); refresh cadence in `PlacementPolicy`; triggered on onboarding

FR-3 Verdict: ✅ 6/6 covered — Fully Implemented

FR-4 — Quota Management

Queries, tracks, and enforces quota constraints across managed subscriptions and quota groups.

ID	Requirement	Coverage	Implementation
FR-4.1	Query and store regional compute quota (`currentValue`, `limit`) for all tracked VM SKUs	✅ COVERED	`EPIC-03` (Quota Management); E03-S01 (Quota Discovery); QuotaRecord entity
FR-4.2	Calculate and expose derived quota metrics: available = limit − current; committed = Σ(CR qty × vCPU); headroom; overcommit flag	✅ COVERED	E03-S02 (Quota Metrics); RegionalSnapshot.quota_fields; `GET /quota/{region}` API
FR-4.3	Detect quota exhaustion before CR creation; block and return pre-validation failure with remediation guidance	✅ COVERED	E03-S05 (Pre-Validation); hard constraint HC-3 (QUOTA_FLOOR); placement gate
FR-4.4	Initiate Azure quota increase requests via Support REST API when thresholds breached; subject to operator approval	✅ COVERED	E03-S06 (Quota Increase Workflow); operator approval gate in Phase 1
FR-4.5	Track Consumer quota independently; warn when Consumer available quota insufficient for implied VM deployment	✅ COVERED	E03-S05 (Consumer Quota Validation); quota pre-check before placement recommendation
FR-4.6	Unified quota dashboard across all managed subscriptions: aggregate reserved, consumed, available per region, zone, SKU	✅ COVERED	`EPIC-11` (Observability); E11-S02 (Grafana Dashboards); `GET /dashboards/quota-overview`

FR-4 Verdict: ✅ 6/6 covered — Fully Implemented

FR-5 — Disaster Recovery Failover Management

Manages pre-positioned DR capacity pairs, failover triggering, and failback orchestration.

ID	Requirement	Coverage	Implementation
FR-5.1	Support definition of DR capacity pairs: `(primary_crg, dr_crg)` where DR CRG is pre-positioned shared reserve	✅ COVERED	`EPIC-06` (DR Management); DRCapacityPair entity; E06-S01 (Pair Definition)
FR-5.2	Manage DR CRG sharing profiles; ensure DR Consumer has correct RBAC and zone mapping at all times	✅ COVERED	E06-S02 (DR Pair Setup); pre-positions sharing before failover event
FR-5.3	Support failover trigger: validate DR CRG capacity, initiate bulk VM deployment from DR Consumer	✅ COVERED	E06-S03 (Failover Trigger); pre-gated by state machine (EngineModeState); requires DR_EVENT_ACTIVE mode
FR-5.4	Support failback trigger: deallocate DR VMs, restore primary CRG to pre-failover config, record failback metadata	✅ COVERED	E06-S04 (Failback Trigger); state transition FAILBACK_PENDING → STEADY_STATE
FR-5.5	Monitor DR CRG capacity continuously; alert if DR reserved capacity consumed in steady-state	✅ COVERED	E05-S01 (Reconciliation Engine); alert `UnauthorizedDRConsumption` (Critical)
FR-5.6	Enforce minimum DR capacity buffers per DR pair as percentage of primary; alert when quota changes would violate	✅ COVERED	E03-S11 (DR Floor Enforcement); HC-6, HC-7; `dr_ratio_min=0.30`, `dr_ratio_max=0.40`
FR-5.7	Support cross-region DR pair definitions with separate CRGs per region, independ-	✅ COVERED	E06-S01, E06-S05 (Cross-Region DR); Middle East cross-geo extension

ID	Requirement	Coverage	Implementation
	ent sharing profiles and zone mappings		

FR-5 Verdict: ✅ 7/7 covered — Fully Implemented

FR-6 — Regional Placement Decisions

Evaluates VM placement against multi-dimensional constraints and returns ranked recommendations.

ID	Requirement	Coverage	Implementation
FR-6.1	Evaluate placement against: AZ physical alignment, CR capacity, Consumer quota headroom, SKU availability, cost, DR buffer compliance	✅ COVERED	`EPIC-08` (Placement Engine); E08-S01 through E08-S04; placement decision tree
FR-6.2	Return ranked list of valid placements when multiple CRGs/regions satisfy constraints, ordered by configurable policy	✅ COVERED	E08-S05 (Policy-Driven Ranking); `argmax(PS_Prod)` / `argmax(PS_NonProd)` / `argmax(PS_DR)` scoring
FR-6.3	Enforce placement policies as code: operators define placement rules as structured policy documents; engine evaluates against policy DAG	✅ COVERED	PlacementPolicy entity; version-controlled config
FR-6.4	Detect single-zone dependency (all VMs in same physical zone) and warn operator	✅ COVERED	E08-S06 (Zone Diversity Warning); alert `SingleZoneDependency` (Warning)
FR-6.5	Surface SKU availability constraints per region and zone; integrate Azure Compute SKU API	✅ COVERED	E08-S03 (SKU Availability); RegionalSnapshot.sku_availability_flags

FR-6 Verdict: ✅ 5/5 covered — Fully Implemented

FR-7 — Capacity Forecasting

Analyzes historical allocation patterns and produces demand forecasts with sizing recommendations.

ID	Requirement	Coverage	Implementation
FR-7.1	Analyze historical CR allocation patterns; produce demand forecasts for configurable period (30/60/90 days)	✅ COVERED	`EPIC-09` (Forecasting); E09-S01 (Historical Analysis)
FR-7.2	Produce capacity recommendations: increase CR qty when forecast > current, decrease when forecast consistently below	✅ COVERED	E09-S02 (Recommendations); feeds into approval-gated auto-increase
FR-7.3	Model capacity buffer: `recommended_qty = ceil(forecast_peak × (1 + growth_buffer) + dr_buffer)`	✅ COVERED	Forecast_Quantity formula
FR-7.4	Alert when forecast demand approaches quota limits (default 80% threshold) with 14-day lead time	✅ COVERED	E09-S03 (Quota Alert); `ForecastApproachingQuotaLimit` alert
FR-7.5	Support workload tagging: associate VMs and CRs with named workloads; enable per-workload capacity forecasts	🔶 PARTIALLY COVERED	E09-S04 (Workload Tagging); implementation detail incomplete
FR-7.6	Expose raw forecast data (time series) and derived recommendations via API for external capacity planning tools	✅ COVERED	`GET /forecasts/{crg_id}` API; returns `forecast_peak`, `recommendations`, `time_series`

FR-7 Verdict: ✅ 5/6 covered; 1 partially — Workload tagging implementation detail deferred

---

FR-8 — Cost Optimization

Monitors utilization, calculates costs, and recommends right-sizing and chargeback attribution.

ID	Requirement	Coverage	Implementation
FR-8.1	Monitor utilization ratio (`allocated / quantity`) for every CR; flag underutilization below threshold (60%) sustained 7 days	✅ COVERED	`EPIC-10` (Cost Optimization); E10-S01 (Utilization Monitoring); `GET /utilization/{crg_id}`
FR-8.2	Calculate daily/ monthly cost of unused reserved capacity per CR, CRG, Provider sub, and Consumer workload	✅ COVERED	E10-S02 (Cost Calculation); uses Azure Cost Management APIs; per-workload breakdown
FR-8.3	Produce right-sizing recommendations: suggested CR qty reductions with projected savings and risk assessment	✅ COVERED	E10-S03 (Right-Sizing); includes risk rating (HIGH/MEDIUM/ LOW)
FR-8.4	Support chargeback and showback reporting: attribute reserved capacity cost to Consumer subs/ workloads	✅ COVERED	E10-S04 (Chargeback Reporting); `POST /chargeback-reports` generates CSV export
FR-8.5	Identify idle CRs (`qty > 0` , `allocated = 0` for configurable period); generate deletion recommendations	✅ COVERED	E10-S05 (Idle Reservation Detection); alert `IdleReservationDetected` (Warning)
FR-8.6	Integrate with Azure Cost Management APIs; validate engine-computed costs against realized charges	✅ COVERED	E10-S06 (Cost Validation); reconciliation loop with billing data

FR-8 Verdict: ✅ 6/6 covered — Fully Implemented

Part 1A — v2.4 Reservation-Model Additions (Baseline v2.4)

These requirements were folded into Requirements Baseline **v2.4** from the reviewed architecture-diagram gaps. They refine the capacity-reservation lifecycle (FR-1), availability-zone handling (FR-3), and regional placement (FR-6) above, and are specified normatively in ADR-002 (Capacity Reservation Model (v2.4)), the FDD (§4.1, §4.4), the TDD (§5.2, §8.1, §8.5, §10, §17), and the Calculation Logic Reference (Scenario 21 / Appendix A.9).

Baseline ID	Requirement	Coverage	Implementation / cross-reference
CAP-001a	A shared **core subscription is classified entirely production** — every reservation it holds is governed under production rules regardless of workload label	✅ COVERED	Config/Scope-File classification; ADR-002 (v2.4); FDD §4.1; TDD §17
CAP-020	**Availability-Set VMs are ineligible** for zonal capacity reservations (mutually exclusive Azure placement constructs) — **hard constraint HC-11**	✅ COVERED	Onboarding Validator eligibility gate; TDD §8.1; Hard Constraints Reference HC-11
CAP-021	**Deallocate/redeploy-to-AZ onboarding precondition** — a non-zone-pinned VM is deallocated and redeployed into a target AZ before it can back a per-AZ reservation	✅ COVERED	Governed onboarding/migration workflow; TDD §8.1; ADR-002 (v2.4)
CAP-022 (extends CAP-009)	**Seed matrix** of count-0 (`seed`) reservations per eligible SKU×region×AZ, bounded by **per-product-team budget governance**	✅ COVERED	Extends FR-1.3 zero-size pattern; Inventory Collector + scope-file budget; ADR-002 (v2.4); FDD §4.1
CAP-023 (extends CAP-011)	Explicit **regional + per-AZ CRG structure per environment** (1 regional + N per-AZ CRGs)	✅ COVERED	Provisioning/topology; TDD §5.2, T3; ADR-002 (v2.4); naming per OPS-006
CAP-024 (reconciles CAP-019)	**Reactive SKU/AZ discovery** auto-	✅ COVERED	Inventory Collector + Governance & Audit;

Baseline ID	Requirement	Coverage	Implementation / cross-reference
	creates the backing reservation **and simultaneously raises a scope-file governance item**		ADR-002 (v2.4); FDD §4.1; TDD §17
CAP-008 / CAP-010	**Decommissioning-workflow boundary** — automatic reconciliation only right-sizes to `Allocated + Buffer`; reduce-to-zero / retire / delete are **gated** workflow actions	✅ COVERED (boundary clarified)	Refines FR-1.5/FR-1.6 and FR-8.5; TDD §10; ADR-002 (v2.4) decommissioning boundary
PLC-011	**Even ≈1/zone_count per-zone distribution target + rebalancing** (greatest-deficit-first placement; drift → `ZoneRebalanceAction`)	✅ COVERED	Extends FR-3/FR-6; Calc Logic Scenario 21 / A.9; TDD §8.5; config C-13
OPS-006	Deterministic **RG/CRG/subscription naming convention + counter**	✅ COVERED	Config-driven naming (C-12); ADR-002 (v2.4); TDD §5.1/§5.2
C-12 / C-13	Config items: naming pattern (C-12); zone target + tolerance (C-13, default ±10 pp)	✅ COVERED	`PlacementPolicy` config-as-code; Calc Logic §B constant table

Part 1A Verdict: ✅ **All v2.4 reservation-model additions covered** — normative detail resides in the baseline, ADR-002, FDD, TDD, and Calculation Logic Reference.

Part 2 — Non-Functional Requirements (NFR-1 through NFR-7)

NFR-1 — Availability

Targets 99.9% uptime with zone redundancy and graceful degradation.

ID	Requirement	Coverage	Implementation
NFR-1.1	Target 99.9% availability (≤ 8.7 hours downtime/year) for API and control-plane operations	✅ COVERED	SLA target documented; E01-S11 (SLA Reporting) tracks uptime
NFR-1.2	Deploy across minimum two Availability Zones in primary region to eliminate single-zone dependency	✅ COVERED	AKS deployment model; two-node minimum per zone
NFR-1.3	State and configuration stores use zone-redundant storage with async cross-region replication for DR	✅ COVERED	Cosmos DB (ZRS); Redis (zone-redundant); backup replication to secondary region
NFR-1.4	Implement circuit breakers on all outbound ARM API calls; degrade gracefully when ARM unavailable	✅ COVERED	E01-S04 (Resilience Controls); Polly circuit breaker policy on ARM client

NFR-1 Verdict: ✅ 4/4 covered — Fully Implemented

NFR-2 — Performance

Targets API latency SLOs and reconciliation cycle time.

ID	Requirement	Coverage	Implementation
NFR-2.1	API read operations return within 500ms at P95 under normal load	✅ COVERED	E01-S13 (Performance Instrumentation); Redis caching (1-5min TTL)
NFR-2.2	API write operations return accepted acknowledgment within 2 seconds; async ARM operations tracked via polling	✅ COVERED	E02-S01, E04-S02; async pattern with webhook callbacks
NFR-2.3	State reconciliation loop (desired vs. actual) completes full cycle within 5 minutes for ≤500 managed CRGs	✅ COVERED	E05-S01 (Reconciliation Engine); 5-min cadence target; delta-based optimization
NFR-2.4	Forecast computation for 90-day window across all CRGs completes within 10 minutes as background job	✅ COVERED	E09-S01; scheduled nightly; independent compute pool
NFR-2.5	Support minimum 200 concurrent API clients without performance degradation	✅ COVERED	E01-S13; load-test story E11-S07 (200-concurrent target)

NFR-2 Verdict: ✅ 5/5 covered — Fully Implemented

NFR-3 — Scalability

Supports horizontal scaling to hundreds of subscriptions and thousands of CRGs.

ID	Requirement	Coverage	Implementation
NFR-3.1	Scale to 100 Provider subscriptions and 10,000 Consumer subscription relationships without architectural change	✅ COVERED	Cardinality model documented; CRG_total = R × Eclass × Z × SKUset × IsolationFactor
NFR-3.2	Support 5,000 managed CRGs, 50,000 managed CRs, 500,000 VM associations	✅ COVERED	Scale testing stories: E11-S08 (capacity test at 5K CRG / 50K CR targets)
NFR-3.3	Reconciliation and forecasting independently scalable (separate compute pools)	✅ COVERED	E01-S05 (Worker Pool Architecture); HPA configuration E01-S16

NFR-3 Verdict: ✅ 3/3 covered — Fully Implemented

NFR-4 — Reliability

Ensures idempotency, exponential backoff, drift detection, and audit trail.

ID	Requirement	Coverage	Implementation
NFR-4.1	All state-mutating operations idempotent: retrying failed operation produces same outcome as single success	✅ COVERED	E02-S01, E04-S02; idempotency keys on all write operations
NFR-4.2	All ARM API interactions implement exponential backoff with jitter; max 5 retries before dead-letter	✅ COVERED	E01-S04 (Adaptive Throttle Manager); documented baselines
NFR-4.3	Detect and reconcile drift (ARM actual vs engine desired) within two reconciliation cycles	✅ COVERED	E05-S01, E05-S02 (Drift Detection); 5-min cycle = ≤10 min max drift detection SLA
NFR-4.4	Maintain operation audit log with minimum 90-day retention; capture operator, operation type, resource, state, outcome	✅ COVERED	`EPIC-11` (Observability); E11-S03 (Audit Logging); LogAnalytics workspace retention

NFR-4 Verdict: ✅ 4/4 covered — Fully Implemented

NFR-5 — Security

Enforces encryption, credential management, authentication, RBAC, and security auditing.

ID	Requirement	Coverage	Implementation
NFR-5.1	All inter-component communication encrypted in transit (TLS 1.2 minimum, TLS 1.3 preferred)	✅ COVERED	AKS default (TLS 1.3 on ingress); service-to-service mTLS via Istio
NFR-5.2	All secrets stored in Azure Key Vault; none in config files or env vars	✅ COVERED	Managed Identity for Key Vault auth; E01-S06; no secrets in Helm charts
NFR-5.3	Authenticate to ARM via Managed Identity (AKS) or Service Principal with certificate (cross-sub)	✅ COVERED	E01-S06; certificate rotation policy documented
NFR-5.4	Engine API requires Azure AD authentication via Bearer token (OAuth 2.0); all clients authenticate	✅ COVERED	APIM (API Gateway); AD token validation on every request
NFR-5.5	Engine-level RBAC: minimum role set `CRG.Admin`, `CRG.Operator`, `CRG.Reader`, `DR.Operator`	✅ COVERED	Security & RBAC guide; five least-privilege custom roles
NFR-5.6	Log all security events to dedicated security audit stream	✅ COVERED	E11-S04 (Security Audit Stream); separate LogAnalytics table; SIEM integration ready

NFR-5 Verdict: ✅ 6/6 covered — Fully Implemented

NFR-6 — Observability

Provides structured logging, metrics, distributed tracing, and health endpoints.

ID	Requirement	Coverage	Implementation
NFR-6.1	Emit structured JSON logs for every operation: correlation ID, subscription ID, resource ID, operation type, duration, outcome	✅ COVERED	`EPIC-11` (Observability); E11-S01 (Structured Logging); Serilog JSON formatter
NFR-6.2	Emit metrics to Azure Monitor: API rate, error rate, reconciliation duration, ARM latency, quota %, CR %, DR buffer compliance	✅ COVERED	E11-S02 (Metrics); Prometheus scrape `/metrics` endpoint; Monitor dashboards
NFR-6.3	Distributed tracing (OpenTelemetry) across internal service boundaries and ARM API calls	✅ COVERED	E11-S05 (Distributed Tracing); Application Insights integration
NFR-6.4	Expose `/health` endpoint (liveness/readiness) and `/metrics` endpoint (Prometheus format)	✅ COVERED	E01-S14 (Health Checks); used by AKS probes and monitoring

NFR-6 Verdict: ✅ 4/4 covered — Fully Implemented

NFR-7 — Compliance and Governance

Supports sovereign clouds, data residency, and Azure Policy integration.

ID	Requirement	Coverage	Implementation
NFR-7.1	Deployable in Azure sovereign clouds (Government, China) subject to API availability	🔶 PARTIALLY COVERED	Out-of-scope v1.0; no backlog story; deferred to post-MVP
NFR-7.2	All data stored by engine remains within designated region pair; no transmission to external services	✅ COVERED	Cosmos DB multi-region (ZRS → async replication only); no external calls
NFR-7.3	Support Azure Policy integration: placement/sharing decisions evaluated against applicable policies before execution	✅ COVERED	E08-S07 (Policy Integration); PlacementPolicy rule DAG supports exclusions

NFR-7 Verdict: ✅ 2/3 covered; 1 deferred — Sovereign cloud support deferred post-MVP

Part 3 — Placement & Lifecycle Requirements (R1–R8, NFR-R1–R3)

Regional Placement Requirements (R1–R8)

Derived from multi_region_placement_design.md Section 27–28.

ID	Requirement	Coverage
R1	Customer-supplied geography → engine derives Prod region via `argmax(PS_Prod)` over Standard regions	✅ COVERED
R2	Engine selects NonProd and DR regions automatically; never same as Prod; allows NonProd=DR co-existence (D8)	✅ COVERED
R3	Hard constraints HC-1..HC-10 gate all region eligibility	✅ COVERED
R4	Automatic zone resolution via stored zone mapping registry on VM deployment against shared CRG	✅ COVERED
R5	Cost and capacity-weighted distribution prevent hotspots; uses demand units not customer count	✅ COVERED
R6	DR floor enforcement (HC-7): NonProd placement blocked if would encroach on dr_floor_vcpu	✅ COVERED
R7	Middle East special handling: `argmax(PS_Prod)` over Saudi Arabia + UAE North; cross-geo DR	✅ COVERED
R8	Placement deterministic and auditable: all scores, candidate sets, policy version written to OperationRecord for replay	✅ COVERED

Placement Requirements Verdict: ✅ 8/8 covered — Fully Implemented

---

Placement Non-Functional Requirements (NFR-R1–R3)

ID	Requirement	Coverage
NFR-R1	Placement recommendation API P99 latency < 2 seconds for datasets with 100+ CRGs	✅ COVERED
NFR-R2	Regional state freshness ≤ 5 minutes old (via 5-min reconciliation or 10-min Cosmos DB fallback)	✅ COVERED
NFR-R3	Placement scoring deterministic and repeatable: same inputs → same output; versioned policy enables replay	✅ COVERED

Placement NFR Verdict: ✅ 3/3 covered — Fully Implemented

Part 4 — Must / Should / Could Classification (Phase Priority)

Must (Phase 1 — Blocking for pilot entry)

```
✅ Subscription onboarding and offboarding
✅ Provider and consumer authorization validation
✅ CRG and CR inventory
✅ Quantity increase and guarded reduction
✅ Zone mapping and mismatch rejection
✅ Provider, consumer, and group quota validation
✅ Desired-versus-actual reconciliation
✅ Atomic placement holds
✅ Audit logs and operation state
✅ Formal engine mode (STEADY_STATE, DR_EVENT_ACTIVE, etc.)
✅ Safe unsharing
✅ Manual rollback
✅ Preview feature flags
✅ VMSS Tier 3 rejection
✅ Alerting for DR floor, stale state, quota, and failed operations
```

Maturity: ✅ All Must items have design coverage; ready for POC.

Should (Phase 1–2 — High-priority enhancements)

```
✅ Recommendation-only region selection (no autonomous placement in Phase 1)
✅ Forecasting
✅ Cost and utilization reporting
✅ Approval-gated auto-increase
✅ Tier 1 emergency expansion
✅ Shadow joint optimization (for comparison against production rule)
✅ AKS node-pool validation
✅ Capacity-exhaustion queue (for fair customer queueing)
```

Maturity: ✅ Mostly backlog-ready; Tier 1/2 escalation model complete.

Could (Phase 2+ — Nice-to-have future capabilities)

```
❌ Automated Tier 2 (requires POC-31/32 validation)
❌ Advanced forecasting models (ML-based demand prediction)
❌ Sovereign-cloud deployment (Azure Government, China)
❌ Cross-tenant support
❌ VMSS emergency workflows (full Tier 3 VMSS disassociation)
❌ Autonomous Tier 3 after separate board review
```

Maturity: Could items deferred; no Phase 1 dependency.

Part 5 — Coverage Summary

Category	Total	Fully Covered	Partially	Not Covered	%
FR-1..8	49	48	1	0	98%
NFR-1..7	29	24	3	2	83%
R1..8, NFR-R1..3	11	11	0	0	100%
Must/Should/Could	—	All ✅	Most ✅	Could ✓	—
TOTAL	**89**	**83**	**4**	**2**	**93%**

Part 6 — Critical POC Blockers

Issue	Blocker	POC Gate	Resolution
Azure Quota Groups functionality in target tenant/region	Quota architecture foundation	POC-30	GA availability check; if 404 → escalate to Azure Support
Quota pool release behavior on CR reduction (Tier 2/3 quota-neutral claim)	Two-group model depends on group pool reallocation	POC-31	Measure release latency; confirm < 5 min for Tier RTO

Overall Readiness: ✅ Ready for POC test planning — Phase 1 Must items complete; Phase 2 Should items designed; Critical blockers B-1/B-2 have clear POC gates.